[Huyen Nguyen] Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

1. The boot process

The role of the bootloader

When a device is powered on, the CPU cannot run the Linux operating system immediately. The bootloader is the first piece of code to execute:

- Initializes basic hardware: Enables clocks, configures DDR RAM, and sets up UART (Serial) communication ports.
- Loads the OS: Reads the kernel file from an SD card, Flash memory, or downloads it via a local network, then loads it into RAM.
- Decompresses: If the kernel file is compressed, the bootloader decompresses it into memory.
- Transfers control: Jumps to the kernel's address in RAM so that Linux takes full control of the system.

The bootloader provides a command-line interface (Shell) that allows users to flash software over the network, read/write RAM memory cells, and test hardware.

It is the very first code executed upon power-up: no bootloader -> no kernel -> no operating system -> nothing at all.

It is the first piece of code that users can modify and customize.

ROM code

The ROM code is a piece of code hardwired directly into the CPU chip by the manufacturer. -> It cannot be modified, deleted, or upgraded.

This code runs immediately when the chip is powered on -> Its purpose is to locate where the bootloader resides (SD card, eMMC chip, Flash memory, or USB) in order to load and execute it.

Embedded Chip Boot Process

When powered on, an ARM chip:

- External DDR RAM: is not yet operational because its clocks and timings have not been configured.
- Internal SRAM (on-chip memory): has a small size (32KB–128KB) but is pre-configured -> the bootloader can be loaded here to run temporarily while initializing the external RAM.
- However, U-Boot has a larger size (500KB–1MB) -> it does not fit into the SRAM.

=> Therefore, the bootloader is divided into 2 stages:

- A compact U-Boot version that fits into the SRAM -> initializes and turns on the DDR RAM.
- The full-featured U-Boot version loaded into the DDR RAM to execute.

2. U-Boot

Relocation Mechanism

- When loaded from the memory card/Flash into RAM, U-Boot initially resides at a temporary RAM address.
- After setting up the RAM, it copies its own code to the official RAM region (the upper region of the RAM) to free up space for the Kernel and Rootfs.

Check and Load Optional Configurations

- U-Boot initializes basic peripherals.
- Loads the uEnv.txt file (if present).

Load Linux Kernel and Device Tree into RAM

- U-Boot loads the operating system kernel file (zImage) into RAM.
- U-Boot loads the hardware description file, Device Tree Blob (*.dtb), into RAM.

Fixup Device Tree and Hand Over Control to the Kernel

3. Communication Channels Between Host and Target (Serial Console & Networking)

During embedded systems development, the Host development machine and the Target embedded board are typically connected via two parallel physical channels:

- Serial Channel (UART): Used for viewing boot logs and command-line control.
- Network Channel (Ethernet): Used for loading the Kernel, Device Tree, and transferring files.

Serial Console Access

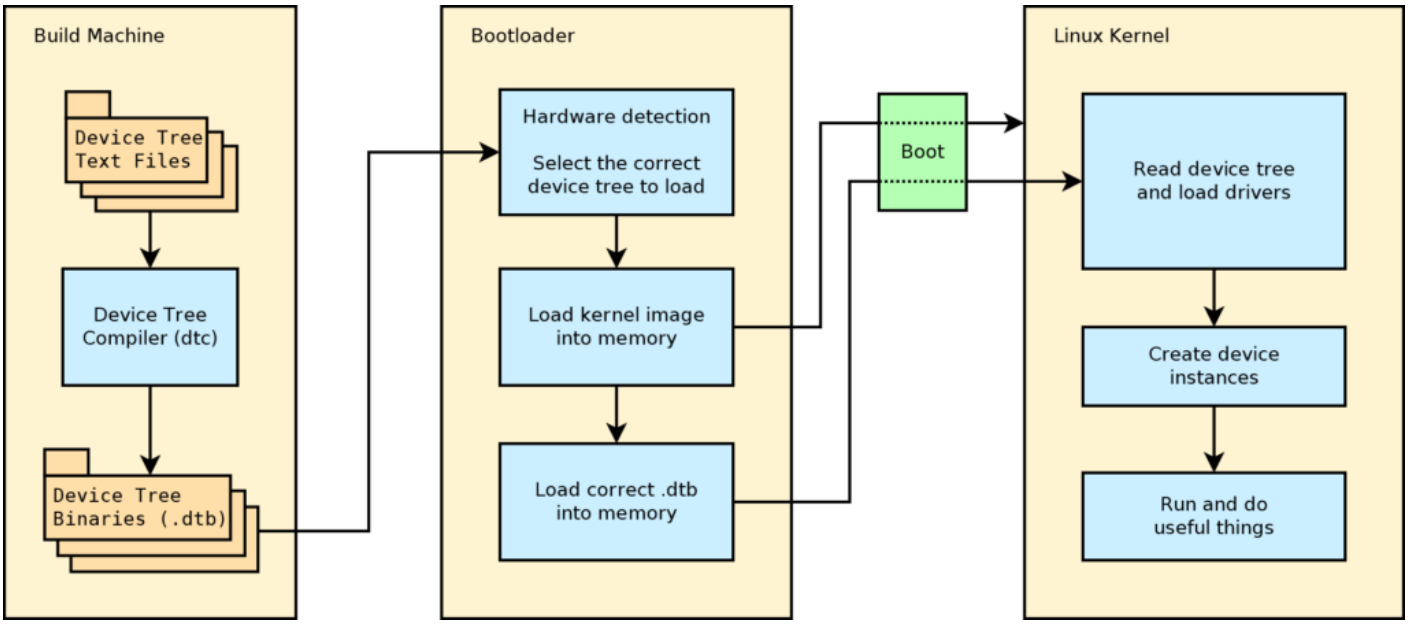
- Uses the asynchronous serial communication standard (UART).
- When the board lacks a display screen, network card, or operating system (running only Boot ROM or U-Boot), UART is essential for observing system execution or debugging errors.

Networking Between Target and Host

LAN (Ethernet):

- Static IP: Configure a fixed IP address for both the Host and Target within the same local network subnet.
- Connection Check: Use the ping command to verify the network cable connection and connectivity to the Host.
- TFTP over UDP: A simple and lightweight file transfer protocol operating at the transport layer using port 69

5. Device Tree



The Linux Device Tree describes the hardware and peripheral devices that the operating system kernel (Kernel image) will interact with during runtime.

The Device Tree Blob (.dtb) binary file is a detailed map of all hardware components mounted on the board that the Kernel cannot discover automatically (non-discoverable hardware)

The .dtb file is a binary file (machine-readable, human-unreadable). To make it human-readable, we use the dtc (Device Tree Compiler) tool to decompile it back into a text-based source file (.dts).

Revision #1
Created 2026-10-22 03:01:50 UTC by Huyen Nguyen
Updated 2026-10-23 08:01:46 UTC by Huyen Nguyen